

Spring Boot Developer Interview Topics

Fresher-level quick revision guide

1. Core Java Basics

OOP Principles

Encapsulation (hide data using private fields + getters/setters), Inheritance (reuse via *extends*), Polymorphism (overloading at compile time, overriding at runtime), and Abstraction (abstract classes and interfaces).

Collections

List (ArrayList, LinkedList) keeps order and allows duplicates; **Set** (HashSet) holds unique items; **Map** (HashMap) stores key-value pairs. HashMap uses hashCode() and equals() to find keys.

Exception Handling

Checked exceptions (IOException) must be handled or declared; unchecked ones (NullPointerException) extend RuntimeException. Use try-catch-finally or try-with-resources.

Java 8 Features

Lambdas, functional interfaces, Stream API (filter, map, collect), Optional to avoid nulls, and default methods in interfaces.

2. Spring Core

What is Spring / Spring Boot?

Spring is a framework for building Java applications using dependency injection. Spring Boot sits on top of it and removes setup work with auto-configuration, starter dependencies, and an embedded server (Tomcat), so an app runs with a single *main* method.

IoC and Dependency Injection

Inversion of Control means Spring creates and manages objects (beans) instead of you using *new*. DI is how those objects receive their dependencies. Constructor injection is preferred because it makes dependencies required and easy to test.

Bean and Bean Scopes

A bean is an object managed by the Spring container. Scopes: **singleton** (default, one instance), **prototype** (new each time), and web scopes **request** and **session**.

Stereotype Annotations

@Component (generic bean), @Service (business logic), @Repository (data access, also translates DB exceptions), @Controller / @RestController (web layer).

@SpringBootApplication

A combination of @Configuration, @EnableAutoConfiguration, and @ComponentScan. It marks the main class and starts scanning from its package.

@Autowired, @Qualifier, @Primary

@Autowired injects a bean. When multiple beans of the same type exist, @Qualifier picks one by name and @Primary marks the default.

3. Spring Boot Essentials

Auto-Configuration

Spring Boot looks at the jars on your classpath and configures beans automatically. For example, adding spring-boot-starter-data-jpa plus a DB driver sets up a DataSource for you.

Starters

Ready-made dependency bundles such as spring-boot-starter-web, -data-jpa, -security, and -test, so you don't manage versions yourself.

application.properties / .yml

Holds configuration like server.port, datasource URL, and logging levels. Read values with @Value or @ConfigurationProperties.

Profiles

Different configs for environments (dev, test, prod) using application-dev.properties and activated by spring.profiles.active.

Actuator

Adds production endpoints like /actuator/health and /actuator/metrics to monitor the app.

4. REST APIs

REST and HTTP Methods

GET (read), POST (create), PUT (full update), PATCH (partial update), DELETE (remove). REST is stateless and resource-based.

Key Annotations

@RestController, @RequestMapping, @GetMapping / @PostMapping etc., @PathVariable (value from URL path), @RequestParam (query parameter), @RequestBody (JSON to object).

ResponseEntity & Status Codes

ResponseEntity lets you control body, headers, and status. Know 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 500 Internal Server Error.

Global Exception Handling

@RestControllerAdvice with @ExceptionHandler methods catches exceptions across all controllers and returns clean error responses.

Validation

Use @Valid on the request body with annotations like @NotNull, @NotBlank, @Size, @Email on DTO fields.

Controller vs RestController

@Controller returns view names (HTML pages); @RestController = @Controller + @ResponseBody, returning data (JSON) directly.

5. Data Access (JPA / Hibernate)

JPA vs Hibernate

JPA is the specification (set of rules); Hibernate is the most common implementation of it. Spring Data JPA adds repositories on top to reduce boilerplate code.

Entity Annotations

@Entity, @Table, @Id, @GeneratedValue, @Column. Relationships: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany.

JpaRepository

Extending JpaRepository gives save, findById, findAll, deleteById for free. Derived queries like findByEmail() are generated from method names; @Query allows custom JPQL/SQL.

Lazy vs Eager Loading

LAZY loads related data only when accessed (default for collections); EAGER loads it immediately. Be ready to explain the N+1 query problem and fixing it with JOIN FETCH.

@Transactional

Runs a method in a single database transaction: all changes commit together or roll back on a runtime exception.

Basic SQL

SELECT, JOINS (inner, left), GROUP BY, HAVING, primary vs foreign keys. Interviewers often ask a simple query like the second-highest salary.

6. Architecture & Security

Layered Architecture

Controller (handles HTTP) -> Service (business logic) -> Repository (database). DTOs carry data between layers so entities aren't exposed directly.

Spring Security Basics

Handles authentication (who you are) and authorization (what you can do). Know the idea of a SecurityFilterChain and password encoding with BCrypt.

JWT

A signed token issued after login and sent in the Authorization header (Bearer token) on each request, enabling stateless authentication.

Microservices (basic idea)

Splitting an app into small independent services that talk over REST. Know the terms: API Gateway, Service Discovery (Eureka), and inter-service calls using Feign or RestTemplate/WebClient.

7. Tools & Testing

Maven / Gradle

Build tools that manage dependencies and packaging. Know pom.xml, and commands like mvn clean install.

Testing

JUnit 5 for unit tests, Mockito to mock dependencies (@Mock, @InjectMocks), @SpringBootTest for integration tests, @WebMvcTest for controller tests.

Lombok

Reduces boilerplate with @Getter, @Setter, @Data, @Builder, @NoArgsConstructor, @AllArgsConstructor.

Git & Postman

Git basics: clone, add, commit, push, pull, branch, merge. Postman for testing API endpoints.

Tip: Build one small CRUD project (e.g., Employee or Student management with REST + JPA + MySQL) and be ready to explain its flow end to end. Most fresher interviews revolve around your own project.